CAST Imaging Backup / Restore Procedure for Kubernetes

Prerequisite and important notes:

This procedure can be used to perform a full backup of a source Imaging 3.4.x instance running on Kubernetes.

The backup files can then be used to restore Imaging in a target instance of same version located.

The name of the Imaging namespace at the source and the target must be identical.

Before proceeding with a backup or restore, the source or target Imaging instance should be quiet (no activity shown in Console).

The backup and restore procedures cover:

- The analysis-node content (uploaded files & source code)
- Cast Storage Service content (built-in postgres instance)
- Neo4j databases

You should have **kubectl** utility installed and configured locally to access the source/target Kubernetes cluster(s).

In this document, we assume that the Imaging namespace is "**castimaging-v3**".

Backup

1. **Backup analysis node(s):**

Connect to the “console-analysis-node-core-0” pod in a shell:

```
kubectl exec -it console-analysis-node-core-0 -n castimaging-
v3  -- /bin/bash
```

Inside the shell, run these commands to create the backup files:

```
tar -czpf /opt/cast/shared/shared-dir.tar.gz --
exclude='/opt/cast/shared/shared-dir.tar.gz' --
exclude='/opt/cast/shared/lost+found' /opt/cast/shared/*
```

```
tar -czpf /usr/share/CAST/cast-dir-0.tar.gz --
exclude='/usr/share/CAST/cast-dir-0.tar.gz' --
exclude='/usr/share/CAST/lost+found' /usr/share/CAST/*
```

Download the backup files locally:

```
kubectl cp castimaging-v3/console-analysis-node-core-
0:/opt/cast/shared/shared-dir.tar.gz ./shared-dir.tar.gz
```

```
kubectl cp castimaging-v3/console-analysis-node-core-
0:/usr/share/CAST/cast-dir-0.tar.gz ./cast-dir-0.tar.gz
```

For each additional analysis node if any, backup their respective /opt/cast/shared folder:

Connect to the “console-analysis-node-core-1” pod in a shell:

```
kubectl exec -it console-analysis-node-core-1 -n castimaging-
v3  -- /bin/bash
```

Inside the shell, run these commands to create the backup files:

```
tar -czpf /usr/share/CAST/cast-dir-1.tar.gz --
exclude='/usr/share/CAST/cast-dir-1.tar.gz' --
exclude='/usr/share/CAST/lost+found' /usr/share/CAST/*
```

Download the backup file locally:

```
kubectl cp castimaging-v3/console-analysis-node-core-
0:/usr/share/CAST/cast-dir-1.tar.gz ./cast-dir-1.tar.gz
```

Continue with “console-analysis-node-core-2” … etc.

2. **Backup CSS Postgres instance (if deployed in the cluster):**

Find the console-postgres pod name:

If you are running kubectl on Linux:

```
kubectl get pods -n castimaging-v3 | grep postgres
```

If you are running kubectl on Windows:

```
kubectl get pods -n castimaging-v3 | findstr postgres
```

In next commands, we will assume that the pod name is “console-postgres-566c8d7cf6-dmnwp”.

Connect to the pod in a shell:

```
kubectl exec -it console-postgres-566c8d7cf6-dmnwp -n
castimaging-v3  -- /bin/bash
```

Inside the shell, run these commands to create the backup files:

```
mkdir /var/lib/postgresql/data/backup

pg_dumpall -U operator -p 5432 -f
/var/lib/postgresql/data/backup/all_databases.backup

exit
```

Download the backup files locally:

```
kubectl cp castimaging-v3/console-postgres-566c8d7cf6-
dmnwp:/var/lib/postgresql/data/backup/all_databases.backup
./all_databases.backup
```

3. **Backup neo4j databases**

Connect to the neo4j pod in a shell:

kubectl exec -it viewer-neo4j-core-0 -n castimaging-v3 -- /bin/bash

Cleanup the backup folder (alternatively, you can move its content to a different location in case you want to keep it):

```
rm /var/lib/neo4j/config/neo4j5_data/backup/*
```

... or create it if not present:

```
mkdir /var/lib/neo4j/config/neo4j5_data/backup
```

Backup the databases:

```
neo4j-admin database backup --verbose --compress=true --
include-metadata=all --pagecache=4G --to-path
/var/lib/neo4j/config/neo4j5_data/backup --from=localhost:6362
"*" > /var/lib/neo4j/logs/backup_ImagingDatabases.log 2>&1
```

Check the backup files:

```
neo4j-admin database backup --inspect-
path=/var/lib/neo4j/config/neo4j5_data/backup
```

You can also review the content of backup_ImagingDatabases.log for any error message.

```
more /var/lib/neo4j/logs/backup_ImagingDatabases.log
```

```
exit
```

Download the backup files locally:

```
kubectl cp castimaging-v3/viewer-neo4j-core-
0:/var/lib/neo4j/config/neo4j5_data/backup ./backup
```

Restore

A freshly installed, empty Imaging instance should be running at the target.

1. **Restore analysis node(s)**

Upload the 2 backup files on the first analysis node “console-analysis-node-core-0”:

```
kubectl cp -n castimaging-v3 ./cast-dir-0.tar.gz console-analysis-node-core-0:/usr/share/CAST/
```

```
kubectl cp -n castimaging-v3 ./shared-dir.tar.gz console-analysis-node-core-0:/opt/cast/shared
```

Connect to the console-analysis-node-core-0 pod in a shell:

```
kubectl exec -it console-analysis-node-core-0 -n castimaging-v3  -- /bin/bash
```

Expand the archives:

```
tar -xzpf /opt/cast/shared/shared-dir.tar.gz -C /
tar -xzpf /usr/share/CAST/cast-dir-0.tar.gz -C /
```

Cleanup:

```
rm /opt/cast/shared/shared-dir.tar.gz
rm /usr/share/CAST/cast-dir-0.tar.gz
```

For each additional analysis node if any, retore their respective cast-dir-x.tar.gz archive:
Connect to the pod in a shell:

```
kubectl exec -it console-analysis-node-core-1 -n castimaging-v3  -- /bin/bash
```

Expand the archives:

```
tar -xzpf /usr/share/CAST/cast-dir-1.tar.gz -C /
```

Cleanup:

```
rm /usr/share/CAST/cast-dir-1.tar.gz
```

Continue with “console-analysis-node-core-2” … etc.

2. Restore the CSS Postgres instance (if deployed in the cluster)

Stop all services except postgres and neo4j:

```
kubectl scale deployment  console-dashboards             --replicas=0 -n castimaging-v3
kubectl scale statefulset console-analysis-node-core     --replicas=0 -n castimaging-v3
kubectl scale deployment  console-service                --replicas=0 -n castimaging-v3
kubectl scale deployment  console-authentication-service --replicas=0 -n castimaging-v3
kubectl scale deployment  console-gateway-service        --replicas=0 -n castimaging-v3
kubectl scale deployment  console-control-panel          --replicas=0 -n castimaging-v3
kubectl scale deployment  console-sso-service            --replicas=0 -n castimaging-v3
kubectl scale deployment  viewer-aimanager               --replicas=0 -n castimaging-v3
kubectl scale deployment  viewer-api                     --replicas=0 -n castimaging-v3
kubectl scale deployment  viewer-etl                     --replicas=0 -n castimaging-v3
kubectl scale deployment  viewer-server                  --replicas=0 -n castimaging-v3
kubectl scale deployment  extendproxy                    --replicas=0 -n castimaging-v3
kubectl scale deployment  mcp-server                    --replicas=0 -n castimaging-v3
```

Find the console-postgres pod name:

If you are running kubectl on Linux:

```
kubectl get pods -n castimaging-v3 | grep postgres
```

If you are running kubectl on Windows:

```
kubectl get pods -n castimaging-v3 | findstr postgres
```

In next commands, we will assume that the pod name is “console-postgres-566c8d7cf6-vdqrs”.

Upload the backup files:

```
kubectl cp -n castimaging-v3 ./all_databases.backup console-postgres-566c8d7cf6-vdqrs:/var/lib/postgresql/data/
```

On **OpenShift**, use this command instead:

```
kubectl cp -n castimaging-v3 ./all_databases.backup console-postgres-566c8d7cf6-vdqrs:/var/lib/pgsql/data/
```

Connect to the pod in a shell:

```
kubectl exec -it console-postgres-566c8d7cf6-vdqrs -n
castimaging-v3  -- /bin/bash
```

Open a psql session:

```
psql -U postgres
```

Execute these commands in psql:

```
drop database keycloak;
drop schema control_panel cascade;
```

Exit psql (CTRL+D, you will be back to the shell) and run these commands to restore the backup:

Restore command:

```
psql -U postgres -f
/var/lib/postgresql/data/all_databases.backup
```

➔ On **OpenShift**, use this command instead:

```
psql -U postgres -f /var/lib/pgsql/data/all_databases.backup
```

Messages 'ERROR: role "xxx" already exists' can be ignored

Run the "analyze" commands:

```
psql -U operator -p 5432 -c "ANALYZE;" -d keycloak
psql -U operator -p 5432 -c "ANALYZE;" -d postgres
```

Exit the Shell (CTRL+D).

3. Restore neo4j databases

Upload the backup files to the neo4j pod:

```
kubectl cp -n castimaging-v3 ./backup viewer-neo4j-core-
0:/var/lib/neo4j/config/neo4j5_data
```

Connect to the neo4j pod in a shell:

```
kubectl exec -it viewer-neo4j-core-0 -n castimaging-v3  --
/bin/bash
```

Open a cypher-shell session:

```
cypher-shell -a localhost:7687 -u neo4j -p imaging -d system
```

Delete and re-create the neo4j, imaging and packagereference databases:

```
drop database neo4j if exists;
drop database imaging if exists;
drop database packagereference if exists;

create database neo4j;
create database imaging;
create database packagereference;

stop database neo4j;
stop database imaging;
stop database packagereference;
```

Disconnect the cypher-shell (CTRL+D). You will be back to the shell inside neo4j pod.

Restore the 3 databases with these 3 commands:

```
neo4j-admin database restore --verbose --overwrite-
destination=true --from-
path=/var/lib/neo4j/config/neo4j5_data/backup --source-
database=neo4j neo4j

neo4j-admin database restore --verbose --overwrite-
destination=true --from-
path=/var/lib/neo4j/config/neo4j5_data/backup --source-
database=imaging imaging

neo4j-admin database restore --verbose --overwrite-
destination=true --from-
path=/var/lib/neo4j/config/neo4j5_data/backup --source-
database=packagereference packagereference
```

Open a cypher-shell session:

```
cypher-shell -a localhost:7687 -u neo4j -p imaging -d system
```

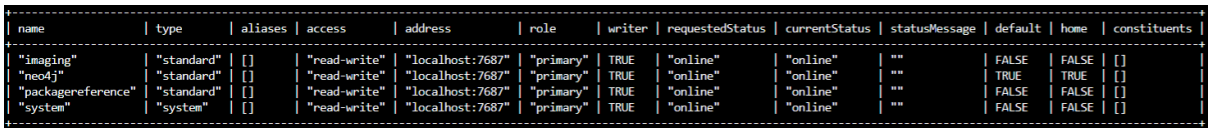
Start the restored databases:

```
start database neo4j;
start database imaging;
start database packagereference;
```

Check databases:

```
show databases;
```

You should see all the restored databases listed online:



name	type	aliases	access	address	role	writer	requestedStatus	currentStatus	statusMessage	default	home	constituents
"imaging"	"standard"	[]	"read-write"	"localhost:7687"	"primary"	TRUE	"online"	"online"	""	FALSE	FALSE	[]
"neo4j"	"standard"	[]	"read-write"	"localhost:7687"	"primary"	TRUE	"online"	"online"	""	TRUE	TRUE	[]
"packagereference"	"standard"	[]	"read-write"	"localhost:7687"	"primary"	TRUE	"online"	"online"	""	FALSE	FALSE	[]
"system"	"system"	[]	"read-write"	"localhost:7687"	"primary"	TRUE	"online"	"online"	""	FALSE	FALSE	[]

Disconnect the cypher-shell (CTRL+D): you will be back to the shell inside neo4j pod.

Execute the 3 permission scripts and exit the shell:

```
cypher-shell -a localhost:7687 -u neo4j -p imaging -d system --param "database => 'neo4j'" -f /var/lib/neo4j/config/neo4j5_data/scripts/neo4j/restore_metadata.cypher
```

```
cypher-shell -a localhost:7687 -u neo4j -p imaging -d system --param "database => 'imaging'" -f /var/lib/neo4j/config/neo4j5_data/scripts/imaging/restore_metadata.cypher
```

```
cypher-shell -a localhost:7687 -u neo4j -p imaging -d system --param "database => 'packagereference'" -f /var/lib/neo4j/config/neo4j5_data/scripts/packagereference/restore_metadata.cypher
```

Exit the Shell (CTRL+D).

Stop neo4j:

```
kubectl scale statefulset viewer-neo4j-core  --replicas=0 -n castimaging-v3
```

Check status:

```
kubectl get statefulset viewer-neo4j-core -n castimaging-v3
```

Wait until command shows “0/0” in READY column:

```
NAME                 READY   AGE
viewer-neo4j-core    0/0     52m
```

Final step

Start all Imaging services:

You execute `Util-ScaleUpAll.bat` / `Util-ScaleUpAll.sh` scripts available in the Imaging helm chart root folder or run helm upgrade:

```
helm upgrade castimaging-v3 --namespace castimaging-v3 .
```